

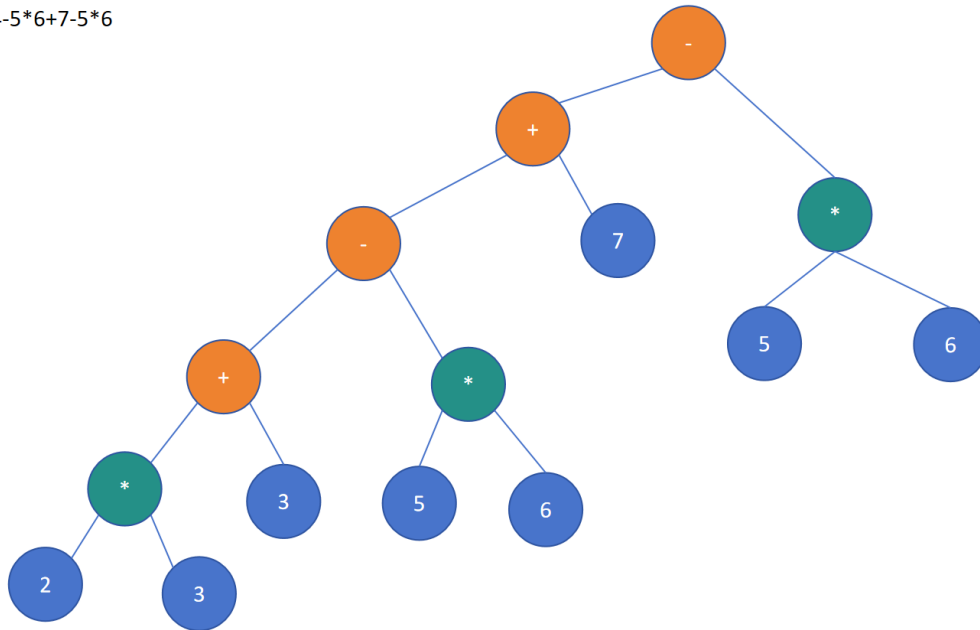
第五章 树的讲义 (3)

这一讲，我们将要讲解二叉树。所引入的案例是一个算式。

$$2 * 3 + 4 - 5 * 6 + 7 - 5 * 6$$

如此构建成的二叉树是：

$2 * 3 + 4 - 5 * 6 + 7 - 5 * 6$



1.构建二叉树

如何构建这个树，我们只能上课讲，写不出来的。大家不妨自己思考思考——我也是自己思考的。

先把代码写出来。

```
1  #include <iostream>
2  using namespace std;
3
4  typedef struct binaryTree{
5      int data;
6      char op;
7      struct binaryTree *left;
8      struct binaryTree *right;
9  } binaryTree;
10
11 /**
12  * @param root 二叉树的头指针
13  * @param treeNode 要增加的节点指针
14  * @return 返回的是head指针——为什么返回，咱们前面讲过了。
```

```

15  */
16  binaryTree* archimedes(binaryTree *root , binaryTree *treeNode) {
17      if (root == nullptr) {
18          root = treeNode;
19          return root;
20      }
21      if (treeNode->op == '#') {
22          binaryTree *p = root;
23          while (p->right != nullptr) {
24              p = p->right;
25          }
26          p->right = treeNode;
27          return root;
28      }
29
30      if (treeNode->op == '+' || treeNode->op == '-' ) {
31          treeNode->left = root;
32          root = treeNode;
33          return root;
34      }
35
36      if (treeNode->op == '*' || treeNode->op == '/' ) {
37          if (root->op == '+' || root->op == '-') {
38              binaryTree *p = root;
39              binaryTree *temp = root;
40              while (p->right != nullptr) {
41                  temp = p;
42                  p = p->right;
43              }
44              temp->right = treeNode;
45              treeNode->left = p;
46              return root;
47          }
48          treeNode->left = root;
49          root = treeNode;
50          return root;
51      }
52  }
53
54  int main() {
55      /**
56       * 我们只考虑一个最简单的情况，一个算式，每个Number都是 个位正整数
57       */
58      // string exp = "2*3+4-5*6+7-5*6";
59      string exp = "2*3*4";
60
61      /**
62       * 在这里解释一下，C语言中，NULL，单单指指针为0，它并非关键字，而是通过
        宏定义实现的。

```

```

63     * #define NULL 0
64     * 然而，宏定义是非常危险的——咱们在课堂上说过这一点。
65     * 因此C++除了一些条件宏定义外，尽量避免用宏，因此它创了一个关键字，
    nullptr
66     * 其含义为：null pointer
67     */
68     binaryTree *root = nullptr;
69
70     for (int i = 0; i < exp.size(); i++) {
71         /**
72         * 第一步：定义一个二叉树节点
73         */
74         binaryTree *treeNode = new binaryTree;
75         treeNode->op = '#';
76         treeNode->data = -1;
77         treeNode->left = nullptr;
78         treeNode->right = nullptr;
79
80         if (exp[i] == '+' || exp[i] == '-' || exp[i] == '*' ||
exp[i] == '/') {
81             treeNode->op = exp[i];
82         }else {
83             treeNode->data = exp[i] - '0';
84         }
85         /**
86         * 第二步：直接将这个节点抛给archimedes函数处理。
87         * archimedes这个名字，大家比较熟悉，就是阿基米德。
88         * 为了构建二叉树，我类比浮力找到了一个规律，
89         * 由于初中物理，浮力和阿基米德几乎是划等号的，所以用了他老人家的
名字
90         */
91         root = archimedes(root, treeNode);
92
93     }
94
95     return 0;
96 }

```

说明一下：

以上代码是我写的，思路与书本上完全不一样。我没有采用DFS算法（深度优先算法）。

第一次按照这个思路写是在JAVA课上，记得有个女生，从这段代码中悟出了“树”概念，或者说，在她的脑海里深刻建立的“树”的模型。所以这一次数据结构课，我依然用这种方式来构建。

这段代码给我的感觉是，要从复杂的环境中找出规律。

用AI构建二叉树，应该来说代码风格更优秀，

用AI写的优秀代码：

```
1  #include <iostream>
2  #include <string>
3  #include <cctype>
4  using namespace std;
5
6  // 二叉树节点定义
7  struct Node {
8      char op;          // 运算符，数字节点为 '#'
9      int val;          // 数字值
10     Node* left;
11     Node* right;
12
13     Node(char o, int v) : op(o), val(v), left(nullptr),
14     right(nullptr) {}
15 };
16
17 // 查找表达式中优先级最低的运算符位置（作为根节点）
18 int findLowestOp(const string& exp) {
19     int pos = -1;
20     int priority = 999;
21     int curPrio;
22
23     for (int i = 0; i < exp.size(); ++i) {
24         if (exp[i] == '+' || exp[i] == '-') curPrio = 1;
25         else if (exp[i] == '*' || exp[i] == '/') curPrio = 2;
26         else continue;
27
28         if (curPrio <= priority) {
29             priority = curPrio;
30             pos = i;
31         }
32     }
33     return pos;
34 }
35
36 // 字符串转数字
37 int strToNum(const string& s) {
38     return stoi(s);
39 }
40
41 // 递归构建表达式二叉树
42 Node* buildTree(const string& exp) {
43     int opPos = findLowestOp(exp);
44
45     // 没有运算符 → 叶子节点（数字）
```

```

45     if (opPos == -1) {
46         return new Node('#', strToNum(exp));
47     }
48
49     // 创建运算符节点
50     Node* root = new Node(exp[opPos], 0);
51
52     // 拆分左右子树
53     string leftExp = exp.substr(0, opPos);
54     string rightExp = exp.substr(opPos + 1);
55
56     root->left = buildTree(leftExp);
57     root->right = buildTree(rightExp);
58
59     return root;
60 }
61
62 // 后序遍历计算表达式值
63 int calcTree(Node* root) {
64     if (!root) return 0;
65
66     // 叶子节点直接返回值
67     if (root->op == '#') return root->val;
68
69     int leftVal = calcTree(root->left);
70     int rightVal = calcTree(root->right);
71
72     switch (root->op) {
73         case '+': return leftVal + rightVal;
74         case '-': return leftVal - rightVal;
75         case '*': return leftVal * rightVal;
76         case '/': return leftVal / rightVal;
77         default: return 0;
78     }
79 }
80
81 int main() {
82     system("chcp 65001");
83     string exp;
84     cout << "请输入无括号表达式（如 3+5*2-8/4）: ";
85     cin >> exp;
86
87     Node* root = buildTree(exp);
88     int res = calcTree(root);
89
90     cout << "二叉树计算结果 = " << res << endl;
91     return 0;
92 }
93

```

2.利用二叉树进行计算

代码如下:

```
1  int calculate(binaryTree *root) {
2      if (root == nullptr) {
3          return 0;
4      }
5      if (root->left == nullptr && root->right == nullptr) {
6          return root->data;
7      }
8      int leftData = calculate(root->left);
9      int rightData = calculate(root->right);
10     if (root->op == '+') {
11         return leftData + rightData;
12     }
13     if (root->op == '-') {
14         return leftData - rightData;
15     }
16     if (root->op == '*') {
17         return leftData * rightData;
18     }
19     if (root->op == '/') {
20         return leftData / rightData;
21     }
22 }
```

看不懂的，上课要认真听讲。